P3 repository

https://github.com/mahamayashen/FlowDay/

claude.md

https://github.com/mahamayashen/FlowDay/blob/main/CLAUDE.md

configure

https://github.com/mahamayashen/FlowDay/blob/main/.claude/settings.json

Claude Code Project Setup

Our CLAUDE.md (68 lines) uses three @import references to keep the main context file concise while making detailed docs available: claude-project-instructions.md (the V1 PRD), docs/DATA_MODEL.md (entity schemas), and docs/CONVENTIONS.md (coding style and testing strategy). The permissions in .claude/settings.json use a read-only-first allowlist (Glob, Grep, Read, git log/diff/status, ruff, pytest) and a denylist blocking destructive operations (rm -rf, sudo, force push, and any Write/Edit to .env files).

Context management — /compact: Using /compact to compress conversation context while preserving the working thread. Claude saves key memories before compressing, so the session can continue without losing important decisions.

```
To save what we need to do for TDD in the memory as well.

  Wrote 2 memories (ctrl+o to expand)

  Saved. Both workflow rules are now in memory and will be recalled in future conversations.

  Ready — your move is to write the failing tests for Cycle 1 and commit RED.

* Baked for 38s

/compact

+ Compacting conversation…
```

Context inspection — /context: Using /context to monitor token consumption. CLAUDE.md uses 780 tokens; memory files and skills are also loaded. Total usage is 24k/200k tokens (12%), leaving ample room for the working session. The breakdown by category (system prompt, memory files, skills, messages, free space) helps decide when to /compact versus /clear.

```
/compact
└ Compacted (ctrl+o to see full summary)
└ Read ../../../.claude/projects/-Users-qinyuan-cs-Vibe-Coding-FlowDay/memory/feedback_tdd.md (40 lines)
└ Read ../../../.claude/projects/-Users-qinyuan-cs-Vibe-Coding-FlowDay/memory/feedback_workflow.md (27 lines)
└ Read .claude/settings.json (42 lines)
└ Read backend/pyproject.toml (50 lines)
└ Read backend/env.example (17 lines)
└ Plan file referenced (~/.claude/plans/agile-scribbling-seal.md)

/context
└ Context Usage
```

```
└ Read ../../../.claude/projects/-Users-qinyuan-cs-Vibe-Coding-FlowDay/memory/feedback_tdd.md (40 lines)
└ Read ../../../.claude/projects/-Users-qinyuan-cs-Vibe-Coding-FlowDay/memory/feedback_workflow.md (27 lines)
└ Read .claude/settings.json (42 lines)
└ Read backend/pyproject.toml (50 lines)
└ Read backend/env.example (17 lines)
└ Plan file referenced (~/.claude/plans/agile-scribbling-seal.md)

/context
└ Context Usage
⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛀ ⛀ ⛁   claude-sonnet-4-6 · 24k/200k tokens (12%)
⛁ ⛁ ⛁ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶
⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   Estimated usage by category
⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   ⛁ System prompt: 6.2k tokens (3.1%)
⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   ⛁ System tools: 8k tokens (4.0%)
⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   ⛁ Memory files: 884 tokens (0.4%)
⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   ⛁ Skills: 929 tokens (0.5%)
⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   ⛁ Messages: 7.8k tokens (3.9%)
⛶ ⛶ ⛶ ⛝ ⛝ ⛝ ⛝ ⛝ ⛝ ⛝   ⛶ Free space: 143k (71.6%)
⛝ ⛝ ⛝ ⛝ ⛝ ⛝ ⛝ ⛝ ⛝ ⛝   ⛝ Autocompact buffer: 33k tokens (16.5%)

Memory files · /memory
└ CLAUDE.md: 789 tokens
└ ~/.claude/projects/-Users-qinyuan-cs-Vibe-Coding-FlowDay/memory/MEMOR95 tokens
.md:

Skills · /skills

User
└ typography: 99 tokens
└ SVG Logo Designer: 90 tokens
└ vercel-react-best-practices: 89 tokens
└ frontend-design: 84 tokens
└ find-skills: 79 tokens
└ web-artifacts-builder: 78 tokens
```

Session resumption — claude --continue: Resuming work on the feature/2-db-setup branch after closing the terminal. The --continue flag restores the previous session's full context. We immediately ran /compact and /context to verify the state before continuing implementation.

```
(vibing) ➜  FlowDay git:(feature/2-db-setup) claude --continue
  ▐▛▜▌   Claude Code v2.1.81
 ▝▜▛▘    Sonnet 4.6 · Claude Enterprise
   ▘▘    ~/cs/Vibe Coding/FlowDay

  ↑ Opus now defaults to 1M context · 5x more room, same pricing

* Conversation compacted (ctrl+o for history)


 /compact
  └ Compacted (ctrl+o to see full summary)

 /context
  └ Context Usage
     ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁   claude-sonnet-4-6 · 24k/200k tokens (12%)
     ⛁ ⛁ ⛁ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶
     ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   Estimated usage by category
     ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   ⛁ System prompt: 6.2k tokens (3.1%)
     ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   ⛁ System tools: 8k tokens (4.0%)
     ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   ⛁ Memory files: 884 tokens (0.4%)
     ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   ⛁ Skills: 929 tokens (0.5%)
     ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   ⛁ Messages: 7.8k tokens (3.9%)
     ⛶ ⛶ ⛶ ⛝ ⛝ ⛝ ⛝ ⛝ ⛝   ⛶ Free space: 143k (71.6%)
     ⛝ ⛝ ⛝ ⛝ ⛝ ⛝ ⛝ ⛝ ⛝   ⛝ Autocompact buffer: 33k tokens (16.5%)

     Memory files · /memory
     └ CLAUDE.md: 789 tokens
     └ ~/.claude/projects/-Users-qinyuan-cs-Vibe-Coding-FlowDay/memory/MEMOR95 tokens
     .md:

     Skills · /skills

     User
     └ typography: 99 tokens
     └ SVG Logo Designer: 90 tokens
     └ vercel-react-best-practices: 89 tokens
     └ frontend-design: 84 tokens
     └ find-skills: 79 tokens
     └ web-artifacts-builder: 78 tokens


❯ 


  ? for shortcuts · hold Space to speak
```

/init output and CLAUDE.md iteration: Running /init after the initial CLAUDE.md was already in place. Claude Code scanned 7 files and suggested additions we had missed: the conda environment activation command, the non-standard dev server port (5060), the AI-TDD commit convention ([#N][RED/GREEN/REFACTOR]), and mutation testing commands. We selectively incorporated these into CLAUDE.md, keeping it under the 200-line best practice.

```
/init

Searched for 1 pattern, read 7 files (ctrl+o to expand)

The existing CLAUDE.md is solid. Here are the improvements I'd suggest adding:

Missing from current CLAUDE.md:

1. Conda environment — vibing must be active before any backend commands work
2. Port 5060 — dev server uses non-standard port
3. Single test command — not documented
4. AI-TDD workflow — the core development methodology (RED/GREEN/REFACTOR, hypothesis, mutmut) isn't in CLAUDE.md
5. 4-phase workflow — Explore→Plan→Implement→Commit per issue is a hard requirement

Here's what I'd add to the existing CLAUDE.md:

● Update(CLAUDE.md)
  └ Added 4 lines
    1 +# CLAUDE.md
    2 +
    3 +This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.
    4 +
      # FlowDay — Claude Code Project Context

● Update(CLAUDE.md)
  └ Added 14 lines, removed 6 lines
```

Skill installation: Using /btw to learn about project-local skill installation. The tdd-strict skill is installed without the -g flag so it only applies to this project. This demonstrates Claude Code's extensibility through community skills.

```
Unknown skill: tdd-strict

● Args from unknown skill: write tests for cycle 1

/btw can i install skills just for the project next time? How to do it?

/btw can i install skills just for the project next time? How to do it?

  Yes — omit the -g flag:

  npx skills add svenja-dev/claude-code-skills@tdd-strict -y

  Without -g, the skill installs into .claude/skills/ in the current project directory instead of ~/.agents/skills/ globally. It's then only available
  when Claude Code is run from that project.

Press Space, Enter, or Escape to dismiss
```

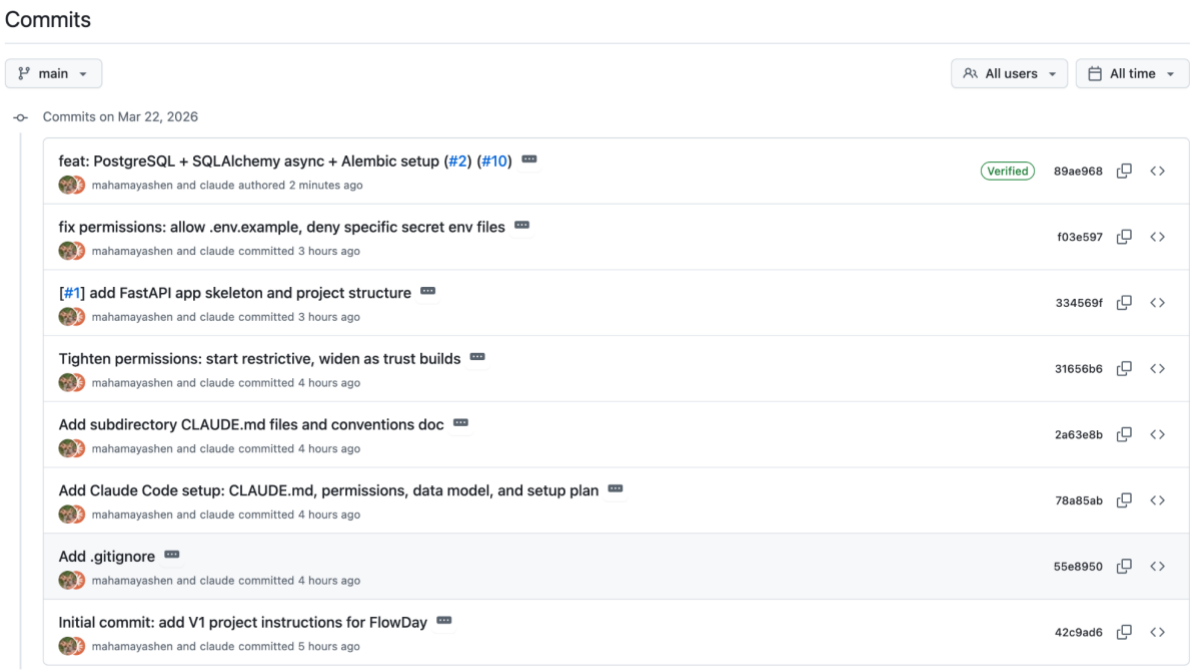
Plan mode exploration: Claude Code in plan mode exploring the CI setup. It detected that no GitHub Actions workflow existed, confirmed that Cycle 1 RED tests could run locally without a database, and proposed a minimal CI workflow. This shows the Explore phase naturally transitioning into Plan.

```
❯ How can I see the tests are failing on GitHub?

● Explore(Check for existing CI configuration)
  └ Done (4 tool uses · 17.3k tokens · 24s)
  (ctrl+o to expand)

  No CI is configured yet — that's why there's nothing to see on GitHub. To see test results there, we need a GitHub Actions workflow.

  Good news: the Cycle 1 RED tests fail because alembic.ini doesn't exist — no database needed. So a minimal CI workflow will show the failures immediately without any
  Docker setup.

● Plan(Plan minimal CI workflow for showing RED tests)
  └ Done (13 tool uses · 17.5k tokens · 1m 33s)
  (ctrl+o to expand)

  There's one issue to fix before the plan is final — the test file has a hardcoded cwd path that will break in CI.

  Read 1 file (ctrl+o to expand)

● Updated plan
  └ /plan to preview

  Ready to code?

  Here is Claude's plan:

  FlowDay — CI Setup + Issue #2 Fix

  Part A: Add GitHub Actions CI (new)

  Context
```

Explore, Plan, Implement, Commit

Git history on main showing the 4-phase workflow

The commit history on main reads bottom-to-top as a clear progression through the 4-phase workflow. The Explore phase happened during “Initial commit” and “Add Claude Code setup” — we read the existing project instructions and understood the architecture before touching any code. The Plan phase produced CLAUDE.md, permissions, and the data model doc. The Implement phase created the FastAPI skeleton ([#1]) and the full database layer (Issue #2, #10) using AI-TDD. The Commit phase shows descriptive, imperative-tense messages with issue references. Each commit was reviewed before merging to main.



TDD with Claude Code

Finding and installing the strict TDD skill

We used /find-skills to search for TDD tooling. Claude Code found three options; we selected svenja-dev/claude-code-skills@tdd-strict because its RED/GREEN/REFACTOR commit convention matched our CLAUDE.md's AI-TDD cycle specification exactly (e.g. "[#2][RED] add failing test: ..."). The skill was installed project-locally (without -g) via npx skills add.

```
* Brewed for 1m 37s

/find-skills

User answered Claude's questions:
└ · What are you looking to find a skill for? → Testing / TDD

● Bash(npx skills find testing tdd 2>&1)
└ npm warn exec The following package was not found and will be installed: skills@1.4.5

  … +31 lines (ctrl+o to expand)

Found several TDD skills. The most relevant for your strict AI-TDD workflow:
```

Skill	Installs	Best for
jwilger/agent-skills@tdd	77	General TDD workflow
svenja-dev/claude-code-skills@tdd-strict	42	Strict TDD (closest to your RED/GREEN/REFACTOR requirement)
proffesor-for-testing/agentic-qe@tdd-london-chicago	63	London/Chicago school TDD styles

```
Given your professor's strict AI-TDD requirement (RED→GREEN→REFACTOR commits, hypothesis, mutmut), tdd-strict sounds most aligned.

npx skills add svenja-dev/claude-code-skills@tdd-strict -g -y

❯ npx skills add svenja-dev/claude-code-skills@tdd-strict -g -y

  ▸▸ accept edits on (shift+tab to cycle)
```

Red-green-refactor commit history on feature/2-db-setup

The feature/2-db-setup branch shows strict TDD discipline across the full database layer. Reading bottom-to-top: each acceptance criterion follows [RED] → [GREEN] → [REFACTOR]. For example, Cycle 1 starts with “[#2][RED] add failing tests: alembic upgrade head runs cleanly,” then “[#2][GREEN] add alembic config + initial migration,” then “[#2][REFACTOR] wire alembic env.py to settings.DATABASE_URL.” Tests were always written before implementation. The branch ends with mutation testing achieving 85% score (11/13 mutants killed), exceeding our 80% target. Ruff lint fixes and hypothesis-based property tests were also integrated into the cycle.

Commits on Mar 22, 2026

[#2][RED] add failing tests: alembic upgrade head runs cleanly mahamayashen and claude committed 1 hour ago	bd85e52
[#2] add GitHub Actions CI and fix hardcoded cwd in migration tests mahamayashen and claude committed 1 hour ago ✗	db195d2
[#2][GREEN] add alembic config + initial migration — upgrade head passes mahamayashen and claude committed 1 hour ago ✗	01e7b6c
fix: update alembic files to use modern Python 3.12 type syntax mahamayashen and claude committed 1 hour ago ✓	8fb5d1c
[#2][REFACTOR] wire alembic env.py to settings.DATABASE_URL — tests s... mahamayashen and claude committed 1 hour ago ✓	d84d266
[#2][RED] add failing tests: get_db yields usable AsyncSession mahamayashen and claude committed 53 minutes ago ✗	7d8eaf8
[#2][GREEN] add database.py with async engine and get_db dep — tests ... mahamayashen and claude committed 52 minutes ago ✗	2022b4c
fix: consolidate hypothesis imports to satisfy ruff I001 mahamayashen and claude committed 48 minutes ago ✗	ead414a
fix: apply ruff auto-fix for import sort in test_db_session.py mahamayashen and claude committed 47 minutes ago ✓	9d7054a
[#2][REFACTOR] move engine into lifespan context manager — tests stil... mahamayashen and claude committed 37 minutes ago ✓	046fd9f
[#2] add mutation tests — 11/13 score (85%) on database.py mahamayashen and claude committed 3 minutes ago ✓	5648d8a